



GRUPO 2

Sistema de control de inventario

//Grupo2.java

Por: Grupo 2



INTEGRANTES

- 01. Bellodas Ramos, Emily Guisell
- 02. Bustos Huayanay, Bruno Renato
- 03. Carbajo Gallegos, Gael Stephan
- 04. Figueroa Estrella, Jhair Alberto
- 05. Jimenes Trujillo, Jack Bryan
- 06. Peña Torres, Adan Imanol
- 07. Saldaña Sanchez, Luis Mario
- 08. Vargas Quispe, Sebastian Alexandre
- 09. Yucra Tintaya, Anthony Cancho



CONTENIDOS

01. Introducción

02. Objetivos

03. Organización del proyecto

04. Descripción de módulos del sistema

05. Estructuras de datos implementadas

06. Funcionamiento general del sistema

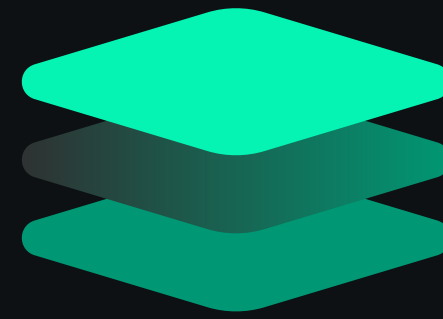
07. Manejo de excepciones

08. Evaluación técnica del proyecto

09. Posibles mejoras

10. Conclusiones

11. Bibliografía



DATOS GENERALES

PROYECTO

Sistema de
Control de
Inventario y
Logística

LENGUAJE

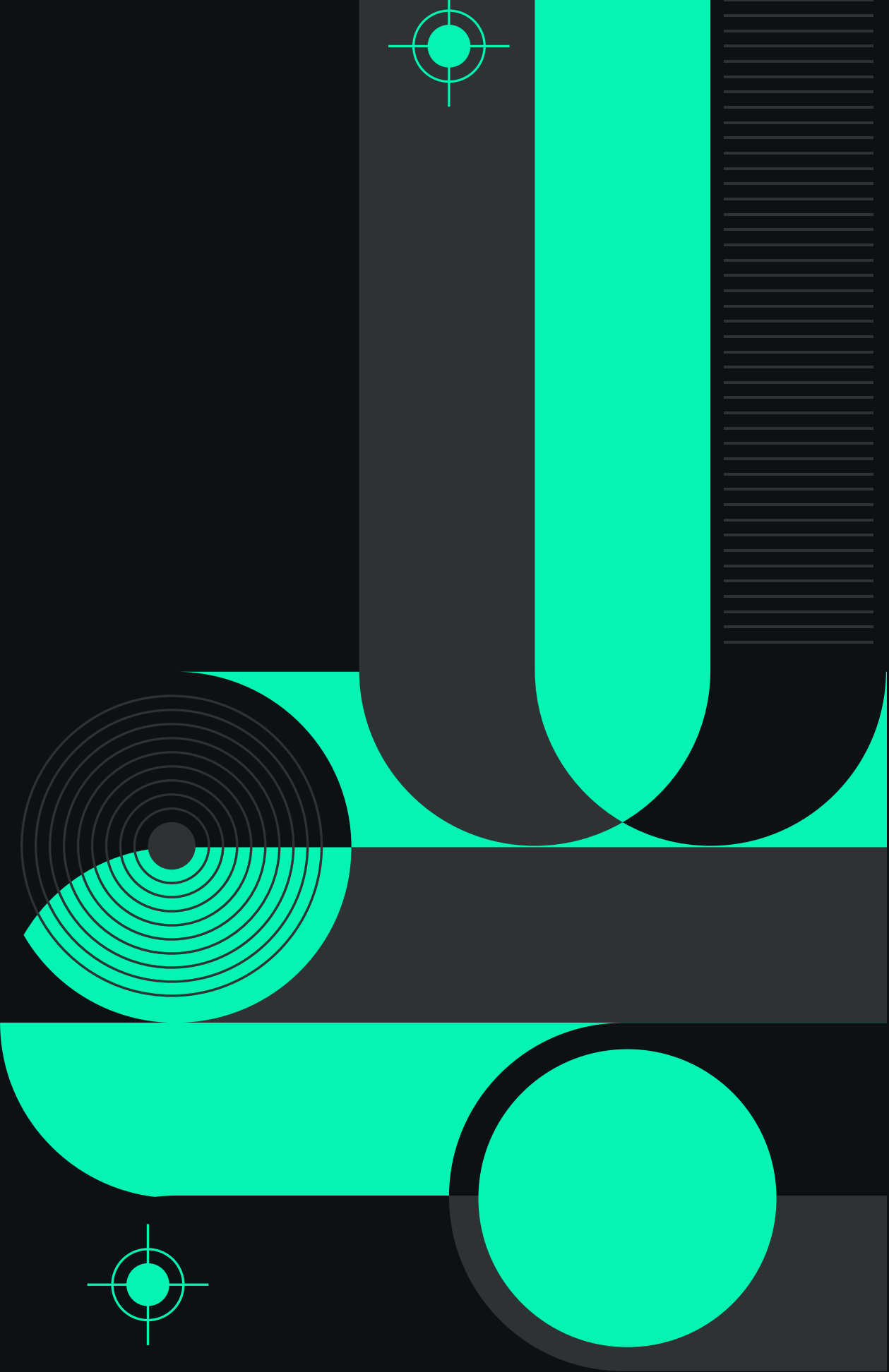
Lenguaje de
programación
java, entorno
NetBeans e
interfaz
gráfica Java
Swing

TIPO DE APLICACIÓN

Aplicación de
escritorio

ESTRUCTURA DE DATOS

- Árbol
Binario de
Búsqueda
- Pila
- Grafo



INTRODUCCIÓN

Proyecto enfocado en la gestión de productos, stock, almacenes, transacciones y rutas logísticas.

Fue desarrollado con una arquitectura modular, separando modelos, servicios, estructuras de datos, excepciones e interfaz gráfica.

Se implementaron estructuras propias como árboles, pilas, listas enlazadas y grafos, aplicando algoritmos para resolver problemas reales y comprender el manejo interno de los datos sin depender de colecciones prediseñadas de Java.

Tecnologías: Java, Estructuras de datos, Algoritmos e interfaces gráficas

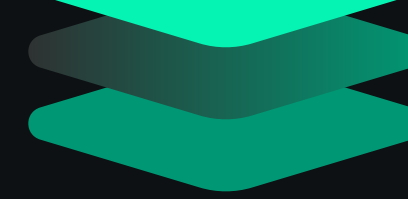
OBJETIVOS DEL PROYECTO

Objetivo General

Desarrollar un sistema de escritorio en Java que permita administrar productos, controlar el stock, registrar movimientos de inventario y gestionar una red logística de almacenes mediante estructuras de datos personalizadas.

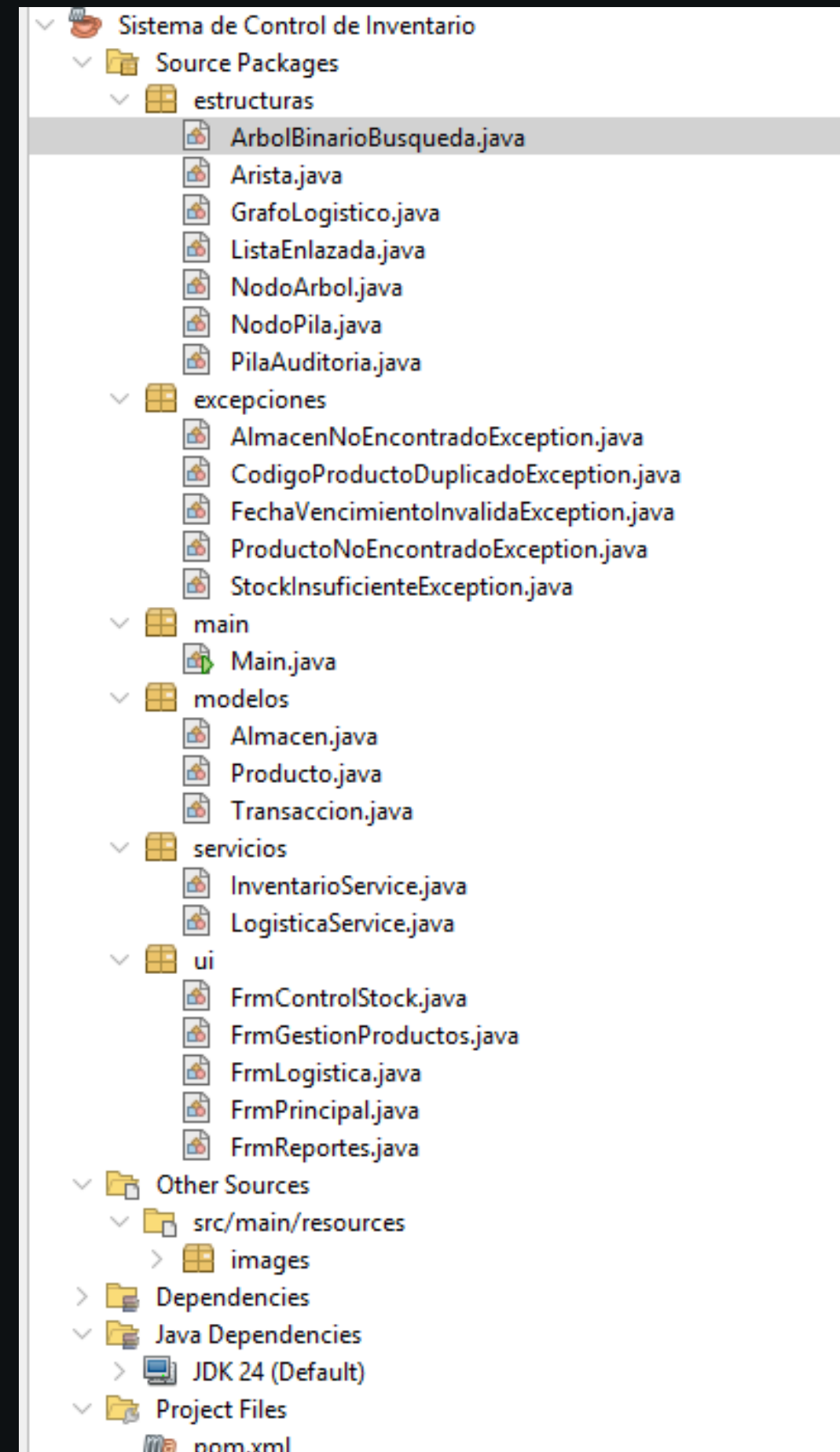
Objetivos Específicos

- Registrar, buscar, modificar y eliminar productos del inventario
- Controlar entradas y salidas de stock.
- Registrar transacciones para mantener un historial de auditoría.
- Administrar almacenes dentro de una red logística.
- Conectar almacenes mediante rutas con distancia en kilómetros.
- Calcular la ruta más corta entre almacenes utilizando un algoritmo de caminos mínimos.
- Aplicar estructuras de datos como árboles, pilas y grafos en un caso práctico.



ORGANIZACIÓN DEL PROYECTO

El proyecto está organizado en paquetes modulares que separan las funcionalidades principales. Incluye estructuras de datos propias, modelos del sistema, servicios de gestión, manejo de excepciones e interfaces gráficas, logrando un código ordenado, mantenible y escalable.



DESCRIPCIÓN DE LOS PAQUETES

4.1 Paquete estructuras

Este paquete contiene las estructuras de datos implementadas para el funcionamiento interno del sistema.

Clase	Descripción
ArbolBinarioBusqueda	Estructura utilizada para organizar y buscar productos.
NodoArbol	Nodo utilizado dentro del árbol binario.
ListaEnlazada	Estructura lineal dinámica utilizada como apoyo para almacenar elementos.
PilaAuditoria	Pila LIFO utilizada para registrar transacciones.
NodoPila	Nodo utilizado internamente por la pila.
GrafoLogistico	Grafo usado para representar almacenes y rutas logísticas.
Arista	Representa una conexión o ruta entre almacenes.

DESCRIPCIÓN DE LOS PAQUETES

4.2 Paquete modelos

El paquete modelos contiene las clases que representan las entidades principales del sistema.

Clase	Descripción
Producto	Representa un producto del inventario.
Almacen	Representa un almacén dentro de la red logística.
Transaccion	Representa una operación o movimiento realizado en el inventario.

4.3 Paquete servicios

El paquete servicios contiene la lógica principal del sistema.

Clase	Descripción
InventarioService	Gestiona productos, stock y transacciones.
LogisticaService	Gestiona almacenes, rutas logísticas y cálculo de caminos.

DESCRIPCIÓN DE LOS PAQUETES

4.4 Paquete excepciones

El paquete excepciones contiene clases utilizadas para controlar errores específicos del sistema.

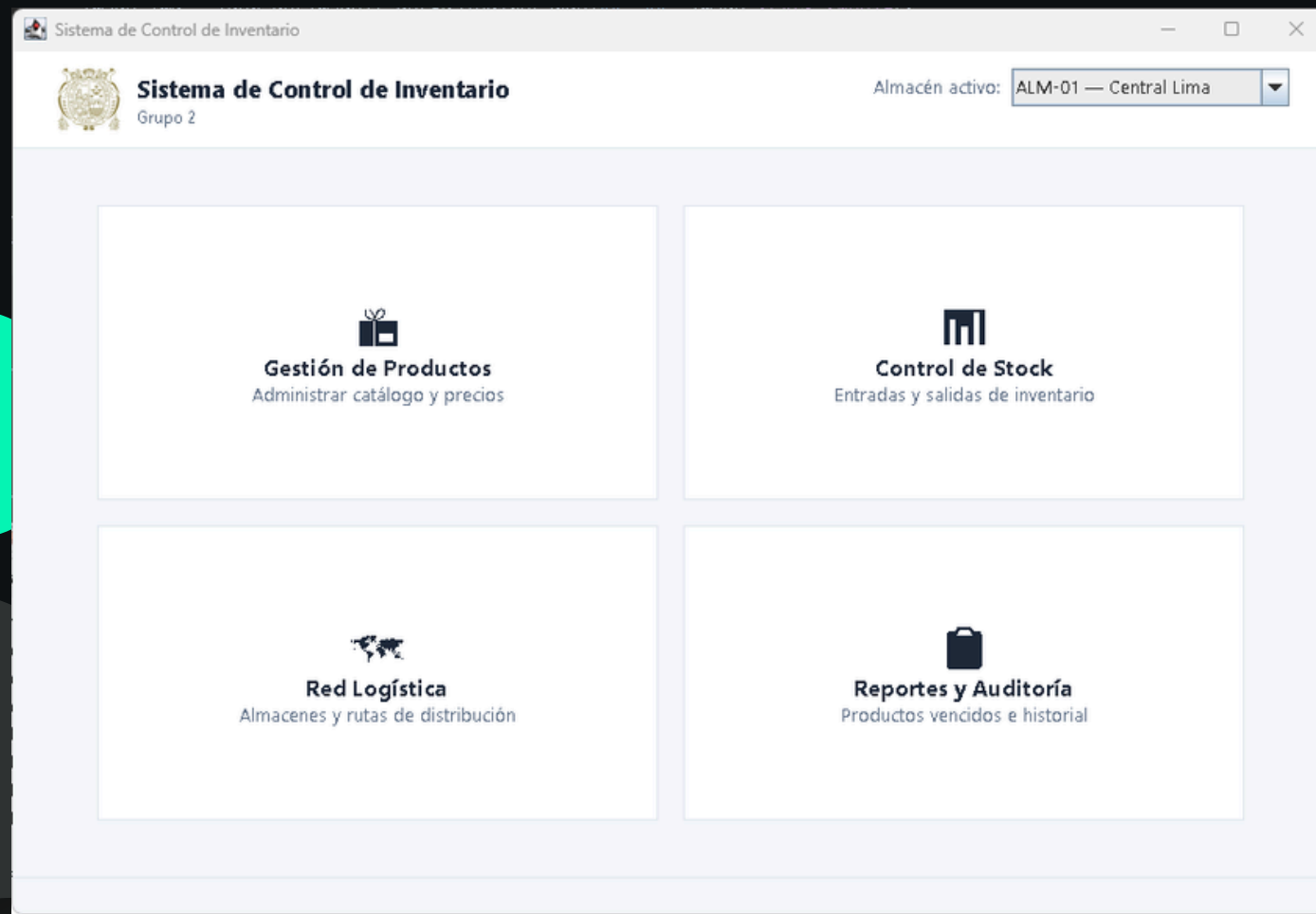
Excepción	Situación en la que se utiliza
CodigoProductoDuplicadoException	Cuando se intenta registrar un producto con un código ya existente.
ProductoNoEncontradoException	Cuando se intenta buscar u operar con un producto inexistente.
StockInsuficienteException	Cuando se intenta retirar una cantidad mayor al stock disponible.
FechaVencimientoInvalidaException	Cuando se ingresa una fecha de vencimiento inválida.
AlmacenNoEncontradoException	Cuando se intenta operar con un almacén no registrado.

El uso de excepciones personalizadas permite que los errores sean más claros y fáciles de manejar.

DESCRIPCIÓN DE MODULOS DEL SISTEMA

El sistema cuenta con diferentes módulos gráficos que permiten al usuario interactuar con sus funcionalidades principales.

- Módulo principal
- Módulo de gestión de productos
- Módulo de control de stock
- Módulo de logística
- Módulo de reportes



Módulo principal

Ventana principal del sistema

El formulario principal permite acceder a los diferentes módulos del sistema. Desde esta ventana el usuario puede abrir la gestión de productos, el control de stock, la logística y los reportes.

DESCRIPCIÓN DE MODULOS DEL SISTEMA

Gestión de Productos

Filtrar por categoría: Todas

Código	Nombre	Categoría	Precio (\$/)	Stock Actual	Stock Min.	Fecha Venc.
ALIM-001	Yogurt Gloria x6	Alimento	\$/ 18.50	170	30	2026-07-23
ALIM-002	Aceite Primor 1L	Alimento	\$/ 9.80	270	50	2028-01-08
ALIM-003	Leche Evaporada ...	Alimento	\$/ 98.40	80	20	2027-07-08
ELEC-001	Laptop Lenovo Ide...	Electronico	\$/ 2500.00	70	10	-
ELEC-002	Monitor Samsung ...	Electronico	\$/ 850.00	25	8	-
FARM-001	Paracetamol 500m...	Farmaceutico	\$/ 12.00	700	100	2028-07-08
FARM-002	Vacuna Influenza (...)	Farmaceutico	\$/ 45.00	8	20	2027-01-08
HERRA-001	Taladro Inalambril...	Herramienta	\$/ 459.00	3	5	-
HOGA-001	Silla Oficina Ergon...	Hogar/Oficina	\$/ 380.00	2	10	-
ROPE-001	Polo Deportivo Ad...	Ropa	\$/ 89.90	185	30	-

Datos del Producto

Tipo: GENERAL

Categoría: Electronico

Código: ELEC-003

Nombre:

Precio (\$/):

Stock Inicial:

Stock Mínimo:

Marca:

Vencimiento:

Registrar

Eliminar

Limpiar

Módulo de gestión de productos
El módulo de gestión de productos permite registrar nuevos productos, buscar productos existentes, modificar información y eliminar registros. Internamente, los productos se organizan utilizando una estructura de árbol binario de búsqueda, lo que facilita su administración mediante el código del producto.

Módulo de control de stock
El módulo de control de stock permite registrar entradas y salidas de productos. Cada movimiento realizado genera una transacción, la cual se almacena en la pila de auditoría del sistema. Esto permite llevar un historial de movimientos, donde la operación más reciente queda disponible en el tope de la pila.

Control de Stock

Filtrar por categoría: Todas

Código	Nombre	Categoría	Stock Actual
ALIM-001	Yogurt Gloria x6	Alimento	170
ALIM-002	Aceite Primor 1L	Alimento	270
ALIM-003	Leche Evaporada Gloria x24	Alimento	80
ELEC-001	Laptop Lenovo IdeaPad	Electronico	70
ELEC-002	Monitor Samsung 24"	Electronico	25
FARM-001	Paracetamol 500mg x100	Farmaceutico	700
FARM-002	Vacuna Influenza (dosis)	Farmaceutico	8
HERRA-001	Taladro Inalambrico Bosch	Herramienta	3
HOGA-001	Silla Oficina Ergonomica	Hogar/Oficina	2
ROPE-001	Polo Deportivo Adidas	Ropa	185

Registrar Movimiento — Almacén: ALM-01

Código:

Cantidad:

Observación:

+ Entrada

- Salida

DESCRIPCIÓN DE MODULOS DEL SISTEMA

Gestión de Red Logística

Nuevo Almacén

ID:

Nombre:

Dirección:

Capacidad:

Tipo:

Agregar Almacén

Conectar Rutas

Origen:

Destino:

Distancia (km):

Descripción:

Crear Ruta

Optimización de Rutas Logísticas

Origen:

Destino:

Calcular Ruta Óptima

Red Logística Actual

RED LOGISTICA - LISTA DE ADYACENCIA

- [ALM-01] Central Lima | CENTRAL | Cap: 50,000 ud.
--(12.5 km)--> [ALM-02] vía Av. Colonial [Hub Callao]
--(25.0 km)--> [ALM-03] vía Av. Universitaria [Regional S/L]
- [ALM-02] Hub Callao | HUB | Cap: 30,000 ud.
--(12.5 km)--> [ALM-01] vía Av. Colonial [Central Lima]
--(18.0 km)--> [ALM-04] vía Panamericana Sur [Tienda Surco]
--(10.0 km)--> [ALM-06] vía Av. Arequipa [Tienda Miraflores]
- [ALM-03] Regional S/L | REGIONAL | Cap: 20,000 ud.
--(25.0 km)--> [ALM-01] vía Av. Universitaria [Central Lima]
--(30.0 km)--> [ALM-04] vía Via Expresa [Tienda Surco]
- [ALM-04] Tienda Surco | TIENDA | Cap: 5,000 ud.
--(18.0 km)--> [ALM-02] vía Panamericana Sur [Hub Callao]
--(30.0 km)--> [ALM-03] vía Via Expresa [Regional S/L]
--(55.0 km)--> [ALM-05] vía Panamericana Sur [Regional Ica]
- [ALM-05] Regional Ica | REGIONAL | Cap: 15,000 ud.
--(55.0 km)--> [ALM-04] vía Panamericana Sur [Tienda Surco]

Eliminar Almacén o Ruta

Almacén:

Eliminar Almacén

Ruta Origen:

Ruta Destino:

Eliminar Ruta

Resultado de Ruta Óptima

Seleccione origen y destino para calcular la ruta óptima.

Módulo de Red Logística

El módulo de logística permite administrar almacenes y rutas entre ellos. Los almacenes se representan como vértices de un grafo y las rutas como aristas con peso, donde el peso corresponde a la distancia en kilómetros.

Este módulo también permite calcular la ruta más corta entre dos almacenes mediante el algoritmo de Dijkstra.

Reportes y Auditoría

Inventario CompletoProductos Perecibles VencidosHistorial de Movimientos

Código	Nombre	Categoría	Tipo	Precio (\$/)	Stock	Stock Min.	Marca	Fecha Venc.
ALIM-001	Yogurt Gloria x6	Alimento	PERECIBLE	\$/ 18.50	170	30	N/A	2026-07-23
ALIM-002	Aceite Primor 1L	Alimento	PERECIBLE	\$/ 9.80	270	50	N/A	2028-01-08
ALIM-003	Leche Evaporada Gl...	Alimento	PERECIBLE	\$/ 98.40	80	20	N/A	2027-07-08
ELEC-001	Laptop Lenovo Ide...	Electronico	GENERAL	\$/ 2500.00	70	10	Lenovo	-
ELEC-002	Monitor Samsung ...	Electronico	GENERAL	\$/ 850.00	25	8	Samsung	-
FARM-001	Paracetamol 500m...	Farmaceutico	PERECIBLE	\$/ 12.00	700	100	N/A	2028-07-08
FARM-002	Vacuna Influenza (...)	Farmaceutico	PERECIBLE	\$/ 45.00	8	20	N/A	2027-01-08
HERRA-001	Taladro Inalambric...	Herramienta	GENERAL	\$/ 459.00	3	5	Bosch	-
HOGA-001	Silla Oficina Ergono...	Hogar/oficina	GENERAL	\$/ 380.00	2	10	Actiu	-
ROPE-001	Polo Deportivo Adi...	Ropa	GENERAL	\$/ 89.90	185	30	Adidas	-

Actualizar

Módulo de Reportes y Auditoría

El módulo de reportes permite visualizar información generada por el sistema, como productos registrados, movimientos de inventario, historial de transacciones o información de la red logística.

ESTRUCTURAS DE DATOS IMPLEMENTADAS

Lista enlazada como estructura auxiliar

La lista enlazada es una estructura de datos lineal que permite almacenar elementos de forma dinámica. Que a diferencia de un arreglo tradicional, una lista enlazada no requiere definir un tamaño fijo desde el inicio.

En el proyecto, la clase ListaEnlazada cumple un rol auxiliar importante, ya que se utiliza para almacenar colecciones internas, como listas de vértices, aristas, caminos o transacciones.

Uso	Descripción
Grafo logístico	Almacena los vértices y las listas de aristas.
Resultado de Dijkstra	Almacena el camino encontrado entre almacenes.
Pila de auditoría	Puede convertir las transacciones a una lista para mostrarlas sin modificar la pila.
Reportes	Permite recorrer información almacenada en estructuras internas.

ESTRUCTURAS DE DATOS IMPLEMENTADAS

Arbol binario de busqueda

En este proyecto, el árbol binario de búsqueda se utiliza para organizar los productos del inventario. La clave principal de comparación es el código del producto, ya que este permite identificar de forma única cada registro.

El árbol binario de búsqueda permite realizar operaciones importantes sobre productos, tales como:

- Registrar productos.
- Buscar productos por código.
- Evitar códigos duplicados.
- Recorrer productos de manera ordenada.
- Apoyar la generación de reportes del inventario.

```
public void insertar(Producto producto) {  
    // [REQUISITO RUBRICA: ÁRBOL BINARIO DE BÚSQUEDA] - inicio de inserción  
    this.raiz = insertarRec(this.raiz, producto);  
    this.tamano++;  
}  
  
public Producto buscar(String codigo) {  
    // [REQUISITO RUBRICA: ÁRBOL BINARIO DE BÚSQUEDA] - inicio de búsqueda  
    NodoArbol resultado = buscarRec(this.raiz, codigo);  
    if (resultado == null) {  
        throw new ProductoNoEncontradoException(codigo);  
    }  
    return resultado.dato;  
}  
  
public ListaEnlazada<Producto> recorrerPreorden() {  
    ListaEnlazada<Producto> lista = new ListaEnlazada<>();  
    preordenRec(this.raiz, lista);  
    return lista;  
}
```

ESTRUCTURAS DE DATOS IMPLEMENTADAS

Pila de auditoría

La pila es una estructura de datos lineal que funciona bajo el principio LIFO, cuyas siglas significan “Last In, First Out”, es decir, “último en entrar, primero en salir”.

En el proyecto, la pila se implementa mediante la clase PilaAuditoria, utilizando nodos enlazados representados por la clase NodoPila. Esta pila almacena objetos de tipo Transaccion, que representan movimientos realizados dentro del inventario.

```
public void push(Transaccion transaccion) {
    // [REQUISITO RUBRICA: PILAS] - Operación PUSH: agrega al tope
    NodoPila nuevoNodo = new NodoPila(transaccion); // Paso 1: crear nodo
    nuevoNodo.siguiente = this.tope;                // Paso 2: enlazar con el tope actual
    this.tope = nuevoNodo;                          // Paso 3: el nuevo nodo ES el nuevo tope
    this.tamano++;
}

//
public Transaccion pop() {
    // [REQUISITO RUBRICA: PILAS] - Operación POP: extrae del tope
    if (estaVacia()) {
        throw new IllegalStateException(
            "No se puede desapilar: la pila de auditoria esta vacia.");
    }
    Transaccion dato = this.tope.dato;              // Paso 1: guardar dato del tope
    this.tope = this.tope.siguiente;                // Paso 2: mover tope al nodo anterior
    this.tamano--;
    return dato;
}

public Transaccion peek() {
    // [REQUISITO RUBRICA: PILAS] - Operación PEEK: consulta sin modificar la pila
    if (estaVacia()) {
        throw new IllegalStateException(
            "No se puede consultar: la pila de auditoria esta vacia.");
    }
    return this.tope.dato;
}
```


ESTRUCTURAS DE DATOS IMPLEMENTADAS

Grafo logístico

La clase GrafoLogistico implementa un grafo no dirigido y ponderado mediante una lista de adyacencia. Cada vértice representa un almacén y cada arista representa una ruta entre dos almacenes.

```
public void conectar(String idOrigen, String idDestino, double distanciaKm, String descripcion) {
    VerticeLogistico vOrigen = obtenerVertice(idOrigen);
    VerticeLogistico vDestino = obtenerVertice(idDestino);

    if (vOrigen == null) throw new AlmacenNoEncontradoException(idOrigen);
    if (vDestino == null) throw new AlmacenNoEncontradoException(idDestino);

    vOrigen.aristas.add(new Arista(idDestino, distanciaKm, descripcion));
    vDestino.aristas.add(new Arista(idOrigen, distanciaKm, descripcion));

    this.numeroAristas++;
}
```

```
public String mostrarRed() {
    if (vertices.isEmpty()) return " La red logistica esta vacia.";

    StringBuilder sb = new StringBuilder();
    sb.append(" RED LOGISTICA - LISTA DE ADYACENCIA\n");
    sb.append(" =====\n");

    for (VerticeLogistico v : vertices) {
        sb.append(String.format(" > [%s] %s | %s | Cap: %d ud.\n",
            v.almacen.getId(), v.almacen.getNombre(), v.almacen.getTipo(), v.almacen.getCapacidadMaxima()));

        if (v.aristas.isEmpty()) {
            sb.append(" (sin conexiones)\n");
        } else {
            for (Arista a : v.aristas) {
                Almacen dest = getAlmacen(a.getIdDestino());
                sb.append(String.format(" %s [%s]\n", a, dest != null ? dest.getNombre() : "?"));
            }
        }
    }

    sb.append(String.format("\n Vertices: %d | Aristas: %d\n", vertices.size(), numeroAristas));
    return sb.toString();
}
```

Operación	Descripción
agregarAlmacen	Agrega un nuevo almacén como vértice.
conectar	Crea una ruta entre dos almacenes.
eliminarAlmacen	Elimina un almacén y sus rutas asociadas.
eliminarRuta	Elimina una conexión entre dos almacenes.
mostrarRed	Muestra la red logística como lista de adyacencia.
listarAlmacenes	Muestra los almacenes registrados.
existeAlmacen	Verifica si un almacén existe.
dijkstra	Calcula la ruta más corta entre almacenes.

ESTRUCTURAS DE DATOS IMPLEMENTADAS

Algoritmo de Dijkstra

El algoritmo de Dijkstra se utiliza para encontrar la ruta más corta entre dos vértices de un grafo ponderado con pesos no negativos. En este proyecto, se aplica para calcular la ruta logística óptima entre dos almacenes.

El peso de cada arista representa la distancia en kilómetros entre almacenes. Por ello, el algoritmo permite determinar cuál es el camino con menor distancia total.

Dato	Función
ids	Almacena los identificadores de los almacenes.
distancias	Guarda la distancia mínima conocida desde el origen.
predecesores	Permite reconstruir el camino final.
visitados	Indica qué almacenes ya fueron procesados.

```
for (Arista arista : vActual.aristas) {
    String idVecino = arista.getIdDestino();
    int v = buscarIndice(ids, idVecino);

    if (v != -1 && !visitados[v] && distancias[u] != Double.MAX_VALUE) {
        double nuevaDist = distancias[u] + arista.getPeso();
        if (nuevaDist < distancias[v]) {
            distancias[v] = nuevaDist;
            predecesores[v] = ids[u];
        }
    }
}
```

FUNCIONAMIENTO GENERAL DEL SISTEMA

Datos del Producto

Tipo:

Categoría:

Código:

Nombre:

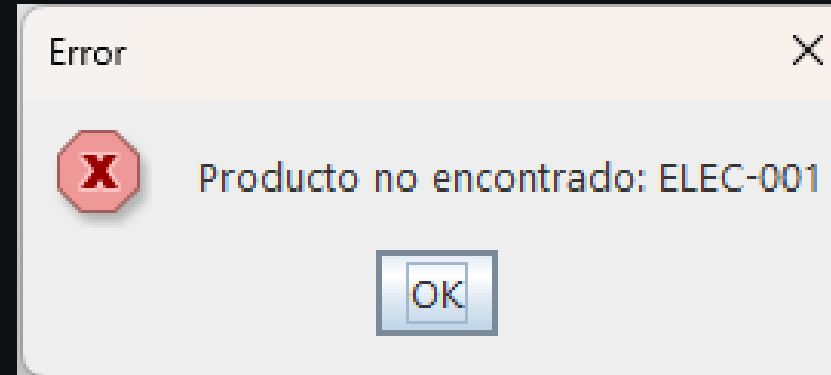
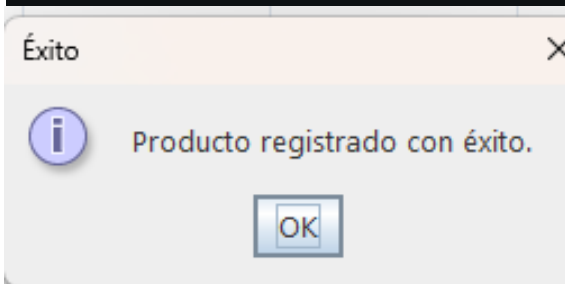
Precio (S/):

Stock Inicial:

Stock Mínimo:

Marca:

Vencimiento:



Filtrar por categoría:

Código	Nombre	Categoría	Precio (S/)	Stock Actual	Stock Mín.	Fecha Venc.
ROPE-001	Polo Deportivo Adidas	Ropa	S/ 89.90	185	30	-

Registrar Movimiento — Almacén: ALM-01

Código: Cantidad: Observación:

Éxito

Entrada registrada correctamente.

ALIM-002	ACEITE PRIMOR 1L	Alimento	270
ALIM-003	Leche Evaporada Gloria x24	Alimento	80
ELEC-001	Laptop Lenovo IdeaPad	Electronico	70
ALIM-003	Leche Evaporada Gloria x24	Alimento	120

Registro del producto

Cuando el usuario registra un nuevo producto, el sistema valida los datos ingresados y luego almacena el producto en la estructura correspondiente. Si el código ya existe, se lanza una excepción personalizada para evitar duplicados.

Busqueda de productos

La búsqueda de productos se realiza mediante el código del producto. La estructura de árbol binario de búsqueda permite ubicar el producto comparando códigos y recorriendo solo las ramas necesarias.

Movimientos de stock

Cuando se realiza una entrada o salida de stock, el sistema actualiza la cantidad disponible del producto. Además, genera una transacción y la almacena en la pila de auditoría.

FUNCIONAMIENTO GENERAL DEL SISTEMA

Red Logística Actual	
RED LOGISTICA - LISTA DE ADYACENCIA	
▶ [ALM-01] Central Lima CENTRAL Cap: 50,000 ud.	
--(12.5 km)-> [ALM-02] via Av. Colonial [Hub Callao]	
--(25.0 km)-> [ALM-03] via Av. Universitaria [Regional SJL]	
--(1232.0 km)-> [ALM-01] via aa [Central Lima]	
--(1232.0 km)-> [ALM-01] via aa [Central Lima]	
--(111.0 km)-> [ALM-01] via aaa [Central Lima]	
--(111.0 km)-> [ALM-01] via aaa [Central Lima]	
▶ [ALM-02] Hub Callao HUB Cap: 30,000 ud.	
--(12.5 km)-> [ALM-01] via Av. Colonial [Central Lima]	
--(18.0 km)-> [ALM-04] via Panamericana Sur [Tienda Surco]	
--(10.0 km)-> [ALM-06] via Av. Arequipa [Tienda Miraflores]	
▶ [ALM-03] Regional SJL REGIONAL Cap: 20,000 ud.	
--(25.0 km)-> [ALM-01] via Av. Universitaria [Central Lima]	
--(30.0 km)-> [ALM-04] via Via Expresa [Tienda Surco]	
▶ [ALM-04] Tienda Surco TIENDA Cap: 5,000 ud.	
--(18.0 km)-> [ALM-02] via Panamericana Sur [Hub Callao]	
--(30.0 km)-> [ALM-03] via Via Expresa [Regional SJL]	
--(55.0 km)-> [ALM-05] via Panamericana Sur [Regional Ica]	
▶ [ALM-05] Regional Ica REGIONAL Cap: 15,000 ud.	
--(55.0 km)-> [ALM-04] via Panamericana Sur [Tienda Surco]	
--(80.0 km)-> [ALM-06] via Carretera Ica-Lima [Tienda Miraflores]	
▶ [ALM-06] Tienda Miraflores TIENDA Cap: 3,000 ud.	
--(80.0 km)-> [ALM-05] via Carretera Ica-Lima [Regional Ica]	
--(10.0 km)-> [ALM-02] via Av. Arequipa [Hub Callao]	
Vertices: 6 Aristas: 9	

Registro de almacenes y rutas

El usuario puede registrar almacenes dentro de la red logística. Posteriormente, puede conectar dos almacenes mediante una ruta con distancia en kilómetros.

Resultado de Ruta Óptima

Ruta Óptima Encontrada

Recorrido: ALM-01 -> ALM-02 -> ALM-04

Distancia Total: 30.50 km

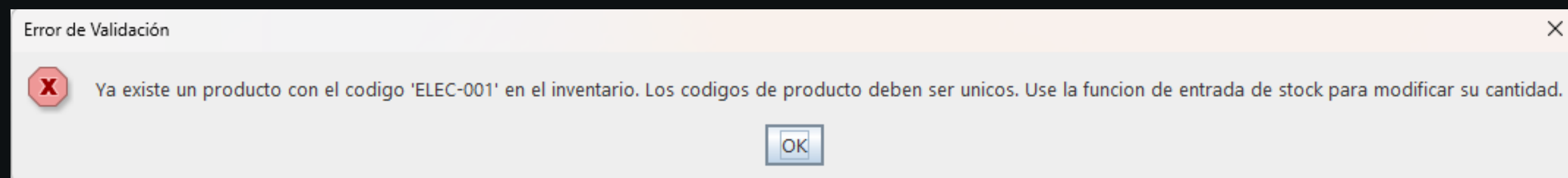
Cálculo de ruta logística

Para calcular la mejor ruta entre dos almacenes, el usuario selecciona un origen y un destino. El sistema aplica Dijkstra y devuelve el camino más corto junto con la distancia total.

MANEJO DE EXCEPCIONES

El sistema utiliza excepciones personalizadas para controlar errores específicos y mostrar mensajes más claros al usuario. El manejo de excepciones permite evitar que el sistema falle de forma inesperada. En lugar de eso, se controla la situación y se informa al usuario sobre el error ocurrido.

Excepción	Descripción
CodigoProductoDuplicadoException	Evita registrar dos productos con el mismo código.
ProductoNoEncontradoException	Informa que el producto buscado no existe.
StockInsuficienteException	Evita retirar más unidades de las disponibles.
FechaVencimientoInvalidaException	Controla fechas de vencimiento no válidas.
AlmacenNoEncontradoException	Informa que un almacén no existe en la red logística.

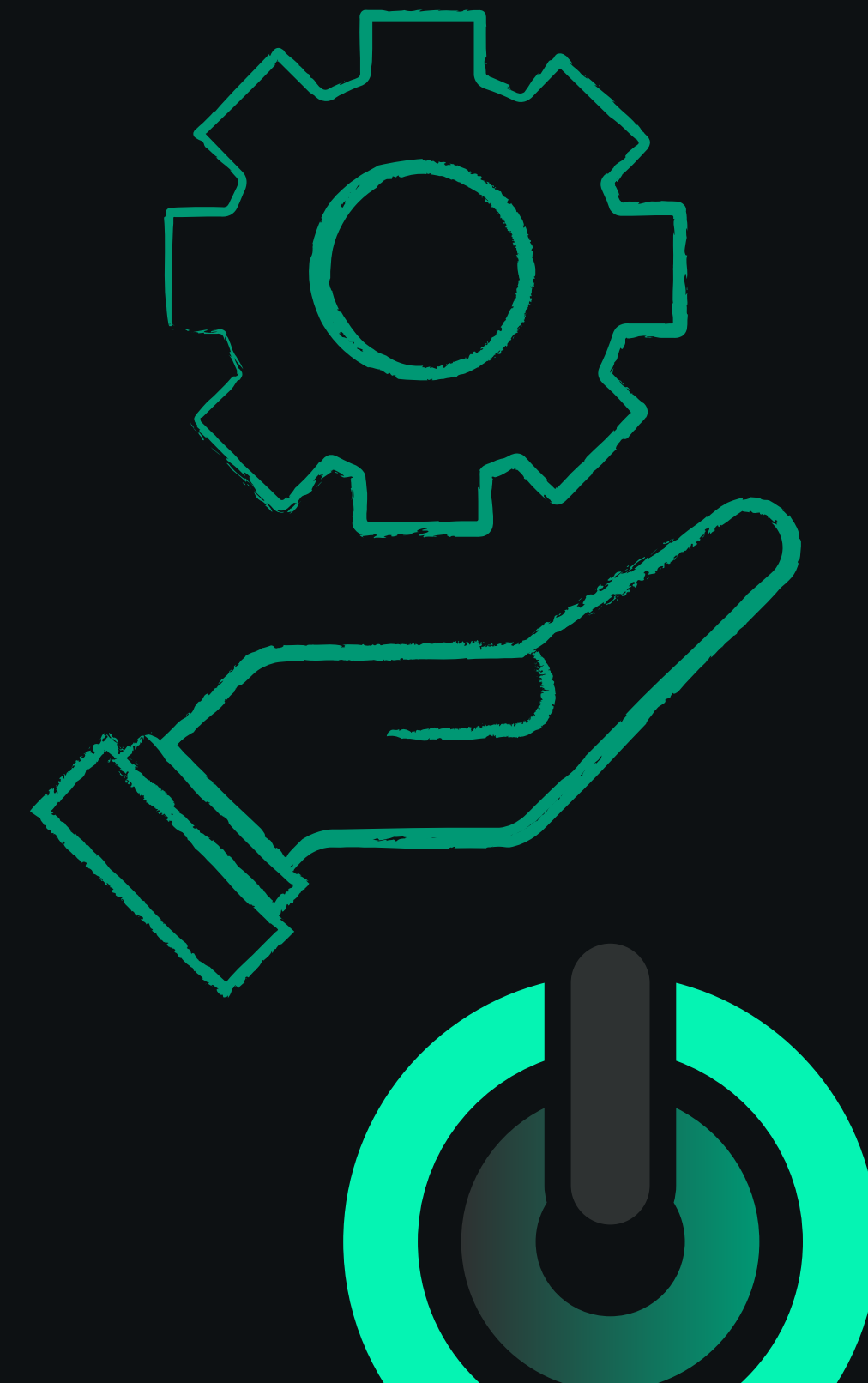


EVALUACIÓN TÉCNICA DEL PROYECTO

El proyecto cuenta con una arquitectura modular que facilita su mantenimiento y organización.

Se separan modelos, servicios, estructuras de datos e interfaces gráficas.

Se implementaron estructuras personalizadas como árboles, pilas y grafos para gestionar productos, auditorías y rutas logísticas. Además, el algoritmo de Dijkstra permite calcular rutas óptimas entre almacenes y las excepciones personalizadas mejoran el control de errores del sistema.

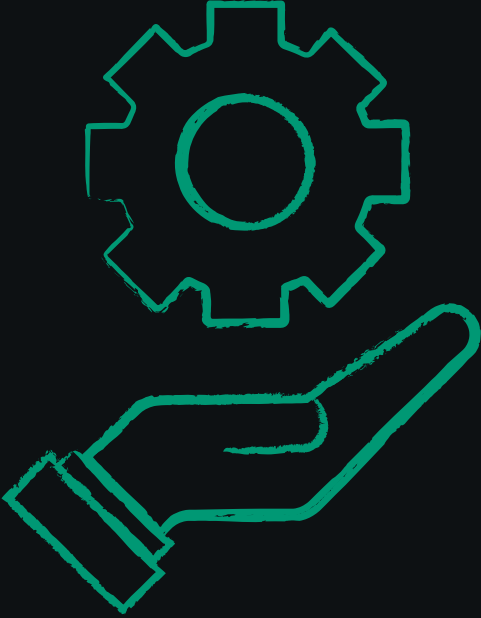


POSIBLES MEJORAS

Mejora	Descripción
Persistencia de datos	Guardar productos, almacenes y transacciones en archivos o base de datos.
Validación de rutas duplicadas	Evitar registrar dos veces la misma conexión entre almacenes.
Validación de distancias	Asegurar que las rutas tengan distancias positivas.
Balanceo del árbol	Implementar un árbol AVL o rojo-negro para mejorar el rendimiento en el peor caso.
Mejora visual de la interfaz	Optimizar el diseño gráfico de los formularios.
Pruebas unitarias	Agregar pruebas automatizadas para validar estructuras y servicios.
Exportación de reportes	Permitir exportar reportes en PDF, Excel u otros formatos.
Gestión de usuarios	Agregar roles como administrador, operador o supervisor.

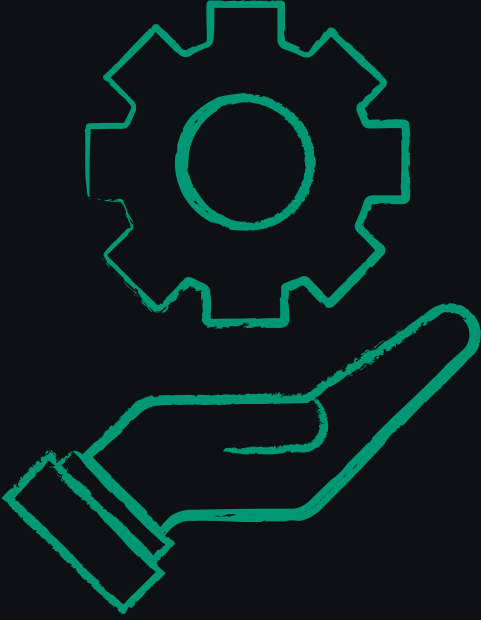
CONCLUSIONES

- El sistema desarrollado integra de forma práctica estructuras de datos y principios de ingeniería de software para la gestión de inventarios y la optimización logística.
- Se implementó un Árbol Binario de Búsqueda para agilizar la organización y consulta de productos, una Pila de Auditoría para registrar operaciones recientes y un Grafo Logístico con el algoritmo de Dijkstra para calcular rutas óptimas entre almacenes.
- El sistema fue diseñado con Programación Orientada a Objetos, una arquitectura por capas, manejo de excepciones e interfaz gráfica en Java Swing, garantizando organización, estabilidad y facilidad de uso.
- En conjunto, el proyecto demuestra la viabilidad de aplicar estas tecnologías en un entorno empresarial y fortaleció el trabajo colaborativo y la integración de los distintos componentes del sistema.



BIBLIOGRAFIA

- Cormen, T., Leiserson, C., Rivest, R. y Stein, C. Introduction to Algorithms.
- Deitel, P. y Deitel, H. Java: How to Program.
- Weiss, M. Data Structures and Algorithm Analysis in Java.
- Documentación oficial de Java.
- Material del curso sobre estructuras de datos, árboles, pilas y grafos.



¡ Gracias !

Email

Grupo2@unmsm.edu.pe

Web

www.Grupo2.es

Teléfono

970 156 521

